

**Федеральное государственное автономное образовательное учреждение высшего
образования «Национальный исследовательский университет ИТМО»**

Факультет программной инженерии и компьютерной техники

ОТЧЁТ
по задачам: A-D
по дисциплине «Алгоритмы и структуры данных»

Выполнил:
Ануфриев Андрей Сергеевич
Р3219

Содержание

1 Задание А. Агроном-любитель

Листинг кода А

```
#include <iostream>
#include <vector>

using namespace std;

int main() {
    int n = 0;
    cin >> n;
    vector<int> a(n);
    for (int i = 0; i < n; i++) {
        cin >> a[i];
    }

    int left = 0;
    int max_len = 1;
    int best_left = 0;
    int count = 1;
    for (int right = 1; right < n; right++) {
        if (a[right] == a[right - 1]) {
            count++;
        } else {
            count = 1;
        }
        if (count == 3) {
            left = right - 1;
            count = 2;
        }
        int current_len = right - left + 1;
        if (current_len > max_len) {
            max_len = current_len;
            best_left = left;
        }
    }
    cout << best_left + 1 << " " << best_left + max_len << endl;

    return 0;
}
```

Суть решения

В решении поддерживается скользящее окно, в котором отслеживается текущая длина подряд идущего одинакового элемента (count).

Когда встречаются три одинаковых подряд, левый край сдвигается так, чтобы в окне осталось не более двух одинаковых подряд.

Для каждого правого конца обновляется длина текущего отрезка и сохраняется лучший найденный отрезок.

Время: $O(n)$ — один проход по массиву.

Память: $O(1)$ — используется константное количество доп. памяти.

2 Задание В. Зоопарк Глеба

Листинг кода В

```
#include <cctype>
#include <iostream>
#include <stack>
#include <string>
#include <vector>

using namespace std;

int main() {
    string s;
    cin >> s;
    int n = s.length() / 2;

    vector<int> result(n, -1);

    stack<tuple<char, int, bool> > st;

    int animal_cnt = 0;
    int trap_cnt = 0;

    for (int i = 0; i < 2 * n; i++) {
        char c = s[i];
        bool is_animal = islower(c) != 0;
        char lower = tolower(c);

        if (!st.empty()) {
            auto [top_char, top_idx, top_is_animal] = st.top();
            if (top_char == lower && top_is_animal != is_animal) {
                int animal_idx, trap_idx;
                if (top_is_animal) {
                    animal_idx = top_idx;
                    trap_idx = trap_cnt;
                } else {
                    animal_idx = animal_cnt;
                    trap_idx = top_idx;
                }
                result[trap_idx] = animal_idx + 1;
                st.pop();
                if (is_animal)
                    animal_cnt++;
                else
                    trap_cnt++;
                continue;
            }
        }

        if (is_animal) {
            st.push({lower, animal_cnt, true});
            animal_cnt++;
        } else {
            st.push({lower, trap_cnt, false});
        }
    }
}
```

```

        trap_cnt++;
    }
}

if (!st.empty()) {
    cout << "Impossible" << endl;
    return 0;
}

cout << "Possible" << endl;
for (int i = 0; i < n; i++) {
    cout << result[i];
    if (i < n - 1)
        cout << " ";
}
cout << endl;

return 0;
}

```

Суть решения

Строка длины $2n$ обрабатывается слева направо; символы двух типов (животные — строчные, ловушки — заглавные) сопоставляются по букве с помощью стека.

В стеке хранятся кортежи (буква в нижнем регистре, индекс в соответствующей группе, `is_animal`).

При встрече символа, если верх стека имеет ту же букву и противоположный тип, происходит пара — формируется соответствие животного и ловушки.

В конце проверяется, пуст ли стек (в противном случае — Impossible).

Время: $O(n)$ — каждый символ обрабатывается один раз.

Память: $O(n)$ — стек и массив результатов длины $O(n)$.

3 Задание С. Конфигурационный файл

Листинг кода С

```
#include <iostream>
#include <stack>
#include <string>
#include <unordered_map>
#include <vector>

using namespace std;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);

    unordered_map<string, long long> vars;
    stack<vector<pair<string, long long> > > restore_stack;

    string line;
    while (getline(cin, line)) {
        if (line == "{") {
            restore_stack.push({});
        } else if (line == "}") {
            auto &top = restore_stack.top();
            for (int i = (int) top.size() - 1; i >= 0; --i) {
                vars[top[i].first] = top[i].second;
            }
            restore_stack.pop();
        } else {
            size_t pos = line.find('=');
            string lhs = line.substr(0, pos);
            string rhs = line.substr(pos + 1);

            bool is_number = !rhs.empty() &&
                (isdigit((unsigned char) rhs[0]) || rhs[0] == '-');

            long long new_val;
            if (is_number) {
                new_val = stoll(rhs);
            } else {
                new_val = vars[rhs];
                cout << new_val << "\n";
                cout.flush();
            }

            if (!restore_stack.empty()) {
                restore_stack.top().push_back({lhs, vars[lhs]});
            }
            vars[lhs] = new_val;
        }
    }
    return 0;
}
```

Суть решения

Поддерживается хеш-таблица текущих значений переменных (`unordered_map`) и стек, в котором для каждого блока

сохраняется вектор пар `{имя, старое_значение}` для восстановления при выходе из блока.

При присваивании константы — просто сохраняем старое значение в текущем блоке (если находимся внутри блока) и обновляем `map`.

При присваивании переменной копируем текущее значение `rhs` и печатаем его (для отладки), затем аналогично сохраняем и обновляем `lhs`. При встрече `}` восстанавливаем значения из вектора в обратном порядке.

тут сам не справился - запутался с вложенностью и выходом, помогала нейронка

Время: $O(m)$ — m строк входа; операции `map` в среднем $O(1)$.

Память: $O(v + b)$ — где v — число различных переменных, b — максимальная глубина/суммарный объём сохранённых записей (в худшем случае $O(m)$).

4 Задание D. Профессор Хаос

Листинг кода D

```
#include <iostream>
#include <map>
#include <vector>

using namespace std;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    long long a, b, c, d, k;
    cin >> a >> b >> c >> d >> k;
    vector<long long> history;
    map<long long, long long> seen;

    long long cur = a;
    for (long long day = 1; day <= k; ++day) {
        if (seen.count(cur)) {
            long long cycle_start = seen[cur];
            long long cycle_len = day - cycle_start;
            long long offset = (k - cycle_start) % cycle_len;
            cout << history[cycle_start - 1 + offset] << "\n";
            return 0;
        }
        seen[cur] = day;
        cur = cur * b;
        if (cur < c) {
            cout << 0 << "\n";
            return 0;
        }
        cur -= c;
        cur = min(cur, d);
        history.emplace_back(cur);
        if (cur == 0) {
            cout << 0 << "\n";
            return 0;
        }
    }
    cout << history.back() << "\n";
    return 0;
}
```

Суть решения

Модель процесса симулируется по дням: на каждой итерации текущее число умножается на b , уменьшается на c , затем ограничивается сверху d . Поскольку k может быть очень большим (до $1e18$), используется обнаружение цикла по состоянию (значению cur): сохраняется отображение $value \rightarrow \text{день}$ и вектор истории.

При повторном появлении состояния вычисляется длина цикла и прямой переход к искомому дню без пошаговой симуляции.

Также обрабатываются случаи завершения эксперимента ($\text{sig}=0$).

пробовал как-то по-умному оптимизировать, но не получилось. В итоге подсказали искать цикл, что он обязательно будет и по нему искать ответ.

Время: $O(\min(k, S))$

Память: $O(\min(k, S))$, где S — число возможных различных состояний (ограничено $d+1$), т.е. алгоритм работает за линейное по числу уникальных состояний время и память.